Assignment – 1

AERT-
Analysis Report

Submitted by - Siddharth Singh Kathait

Roll No. – 2501730320

Course – B.Tech. CSE(AIML)

Subject Code-ETCCDS202

Table of Contents

- About Time Complexity
- Complexity of Functions
    - i. Factorial Function
    - ii. Fibonacci Function
    - iii. Tower of Hanoi Function
    - iv. Binary Search Function
- Paradigm Reflection

Time Complexity Explained Simply

Time complexity tells us how long our code takes when we give it bigger inputs. Think of it like this: "If I double my data, does my program get 2x slower, 4x slower, or 1,000x slower?"

Time Complexity of Functions

Algorithm	Time	Space	Pattern	Use When
Stack	O(1)	O(n)	ADT	Always!
Factorial	O(n)	O(n)	Linear	Small n
Fibonacci	O(2^n)	O(n)	Tree	Learning only
Hanoi	O(2^n)	O(n)	Divide/Conquer	Puzzles
Binary Search	O(log n)	O(log n)	Divide/Conquer	Sorted data

Complexity of Factorial Function

The recursive factorial function has **O(n) time complexity** because it makes exactly *n* recursive calls, each doing one multiplication. Picture it like climbing down a ladder: factorial(5) calls factorial(4), which calls factorial(3), and so on until factorial(1) - that's 5 steps total. Each step takes constant time (just multiply and subtract 1), so total work = n × constant = **O(n)**. It's not bad - doubles your input, doubles your time - but watch the **O(n) space complexity** too, since Python stacks all n calls deep before unwinding back up with the answer. Perfect example of **linear recursion**: simple, predictable, but uses stack space equal to input size.

Complexity of Fibonacci Function

The recursive Fibonacci function has **O(2^n) time complexity** - a classic disaster of **tree recursion**. Unlike factorial's single chain of calls, fib(n) spawns **two branches** every step: fib(n-1) + fib(n-2). This creates an **exploding binary tree** where fib(5) needs fib(4) + fib(3), but fib(4) needs fib(3) + fib(2) - massive **redundant work**! By level *n*, you have roughly **2^n calls**, each doing constant addition. For fib(40), expect **billions of operations** and likely a stack overflow. It's **exponentially slow**: double *n*, quadruple your wait time. Great for teaching recursion's beauty, terrible for real programs - use iteration or memoization instead.

Complexity of Tower of Hanoi Function

The Tower of Hanoi recursive function has **O(2^n) time complexity** - pure exponential disaster! Each call for n disks spawns **two recursive calls** for n-1 disks plus one disk move, creating a binary tree of calls that explodes: 3 disks = 7 moves, 4 disks = 15 moves, 10 disks = 1,023 moves! The recurrence T(n) = 2×T(n-1) + 1 solves to T(n) = 2^n - 1, meaning work **doubles every time you add one disk**. This divide-and-conquer pattern looks elegant but becomes computationally impossible beyond tiny inputs - perfect for teaching recursion's power and peril, but never for real problems. Space complexity O(n) from the recursion stack is the least of your worries when time goes exponential.

Complexity of Binary Search Function

The recursive binary search function achieves **O(log n) time complexity** because it elegantly **halves the search space with every recursive call**. The recursive binary search function achieves **O(log n) time complexity** because it elegantly **halves the search space with every recursive call**.

Paradigm Reflection

Factorial
Paradigm: Linear Recursion (tail recursion)
Each call solves a smaller version of itself: n! = n × (n-1)!. Single chain of calls.
Real Life: Calculating compound interest - Interest on principal + interest on previous interest.

Fibonacci
Paradigm: Tree Recursion (binary branching)
Each call spawns TWO subcalls: F(n) = F(n-1) + F(n-2). Exponential call tree.
Real Life: Rabbit population growth - Each pair produces another pair every month.

Binary Search
Paradigm: Divide and Conquer
Splits problem in HALF each step, discards useless half. Perfect logarithmic efficiency.
Real Life: Phone book lookup - Open middle, pick half, repeat until found.

Tower of Hanoi
Paradigm: Divide and Conquer (3-step recursion)
To move n disks: (1) move n-1 away, (2) move largest, (3) move n-1 on top.
Real Life: Robotic arm stacking - Factory robots moving palettes between conveyor belts.

Stack ADT
Paradigm: LIFO Data Structure (supports recursion)
Last In, First Out - perfect for recursion call stack.
Real Life: Browser back button - Remembers pages in reverse order you visited.